

Lists Introduction

Learning Objectives

- Define what a list is and explain how it differs from arrays
- Create lists, access elements, and modify list contents dynamically
- Identify situations where lists are preferred over arrays

Engage (5-7 minutes)

An array is like a row of fixed mailboxes—each slot holds one item. A list is like a backpack—you can add items, remove items, and the bag adjusts its size automatically. Lists are dynamic, meaning they grow and shrink as needed. Today we learn about this flexible data structure and why modern programming relies on it heavily.

Explore (10-12 minutes)

Create a list of your five favorite subjects. Add “Art” to the end, insert “Music” at position 2, and remove the first item. Now try creating an empty list and adding ten numbers to it one by one. Compare this experience to working with fixed-size arrays: which feels more natural? What happens to the indices when you insert or delete elements?

Explain (10-12 minutes)

A list is a dynamic, ordered collection of elements that can grow or shrink during program execution. Unlike arrays, lists do not require a predefined size. Common operations include append (add to end), insert (add at position), remove (delete by value), pop (delete by index), and len (get size). Lists maintain insertion order and allow duplicate values. In Python, lists are mutable, meaning you can change their contents after creation. Lists use contiguous memory with automatic resizing, making them versatile for most programming tasks.

Elaborate (8-10 minutes)

Create a list of 20 random numbers between 1 and 100. Write functions to: find all even numbers and store them in a new list, remove all duplicates while preserving order, rotate all elements one position to the right, and merge two sorted lists into one sorted list. Analyze how each operation affects the list size and element positions. Discuss with a class: what are the performance implications of inserting at the beginning versus the end of a list?

Evaluate (5-8 minutes)

Given the list [10, 20, 30, 40, 50], write the result of each operation: `append(60)`, `insert(1, 15)`, `remove(30)`, `pop(0)`, `pop()`. Then explain three scenarios where a list is a better choice than a fixed-size array.